

Math Foundations of Machine Learning

Convolutional Neural Networks

Gabriel Wendell Celestino Rocha

July 2025

Table of Contents

Convolutional Neural Networks

- Local Receptive Fields

- Shared Weights and Biases

- Pooling

Cluster Analysis

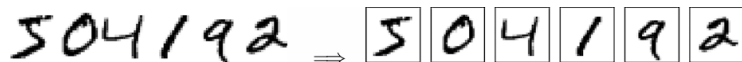
- Basics for Cluster Analysis

- Quality of Clustering

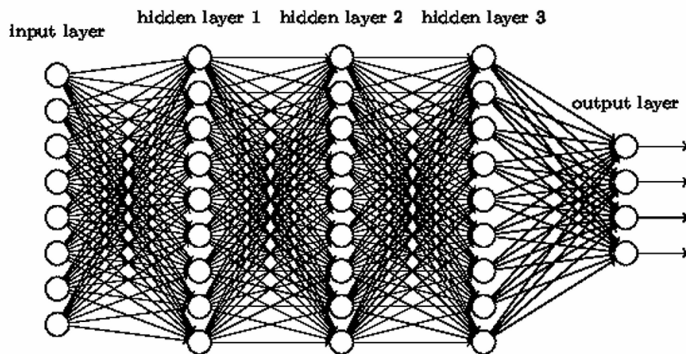
Convolutional Neural Networks

Just a Recap...

Consider neural networks for the classification of handwritten digits, as shown in the following images:



A neural network can be built, with three hidden layers, as follows:



Just a Recap...

- Let each of hidden layers have 30 neurons:
 - $n_weights = 28^2 \cdot 30 + 30 \cdot 30 + 30 \cdot 30 + 30 \cdot 10 = 25,620$
 - $n_biases = 30 + 30 + 30 + 10 = 100$
- Optimization is difficult
 - The number of parameters to teach is huge (**low efficiency**)
 - Multiple local minima problem (**low solvability**)
 - Adding hidden layers is **not necessarily improving** accuracy

In **fully-connected networks**, **deep** neural networks have been **hardly practical**, except for some special applications.

Just a Recap...

Just a Remark...

The neural network exemplified in Slide 3 can produce a **classification accuracy better than 98%**, for the MNIST handwritten digit dataset.

- But upon reflection, it's strange to use networks of fully connected layers to classify images.
 - The network architecture does not take into account the **spatial structure of the images**.
 - For instance, it treats **input pixels** which are far apart and close together, **on exactly the same footing**.

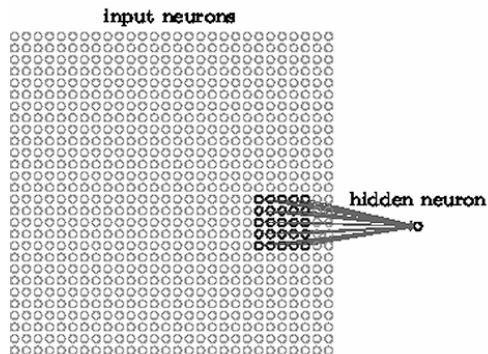
What if, instead of starting with a network architecture which is tabula rasa (blank mind), we use an architecture which tries to **take advantage of the spatial structure**? Could it be **better than 99%**?

Introducing Convolutional Networks

Here, we will introduce **convolutional neural networks (CNN)**, which use a special architecture which is *particularly well-adapted to classify images*.

- The architecture makes **the convolutional networks fast to train**.
- This, in turn, **helps train deep, many-layer networks**.
- Today, deep CNNs or some close variants are used in most neural networks for **image recognition**.
- CNNs use three basic ideas:
 - (a) **Local receptive fields**
 - (b) **Shared weights and biases, &**
 - (c) **Pooling**

Local Receptive Fields



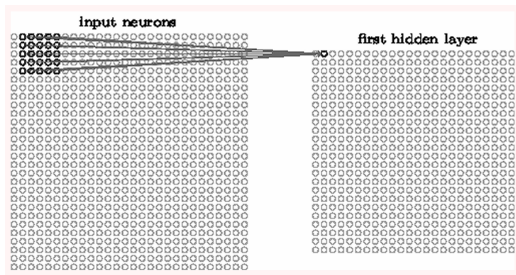
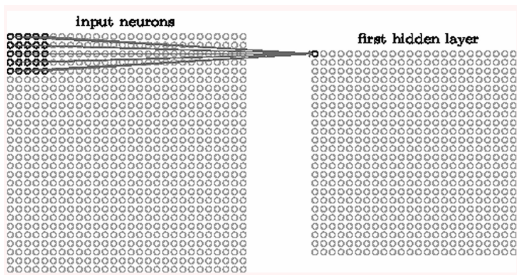
- In CNNs, **the geometry of neurons (units) in the input layer** is exactly the same as that of images (e.g., 28×28 , rather than a vertical line of neurons as in fully-connected networks)

Local Receptive Fields

- As per usual, we'll **connect** the input neurons (pixels) to a layer of hidden neurons
 - But we will **not connect fully** from every input pixel to every hidden neuron.
 - Instead, we only make connections in **small, localized regions** of the input image.
 - **For example:** Each neuron in the first hidden layer will be connected to a small region of the input neurons, say, a 5×5 region.
- That region in the input image is called the **local receptive field** for the hidden neuron.

Local Receptive Fields

- We slide the local receptive field across the entire input image.
 - For each local receptive field, there is a different hidden neuron in the first hidden layer.



Two of local receptive fields, starting from the top-left corner
(Geometry of neurons in the first hidden layer is 24×24).

Local Receptive Fields

Note about the Local Receptive Fields

We have seen that the local receptive field is moved by one pixel at a time (`stride_length = 1`).

- In fact, sometimes a different **stride length** is used.
 - For instance, we might move the local receptive field 2 pixels to the right (or down).
 - Most software gives a hyperparameter for the user to set the stride length.

Shared Weights and Biases

Recall that each hidden neuron has **a bias** and 5×5 **weights** connected to its corresponding local receptive field.

- In CNNs, we use the same weights and bias for each of the 24×24 hidden neurons. In other words, for the $(j, k)^{\text{th}}$ hidden neuron, the output is:

$$\sigma \left(b + \sum_{p=0}^4 \sum_{q=0}^4 w_{p,q} a_{j+p, k+q} \right),$$

where σ is the neural activation function (e.g., the sigmoid function), b is the shared value for the bias, and $w_{p,q}$ is 5×5 array of shared weights.

- The weighting in the equation above is just a form of convolution; we may rewrite it as:

$$\mathbf{a}^1 = \sigma (b + \mathbf{w} * \mathbf{a}^0)$$

- So the network is called a **convolutional network**.

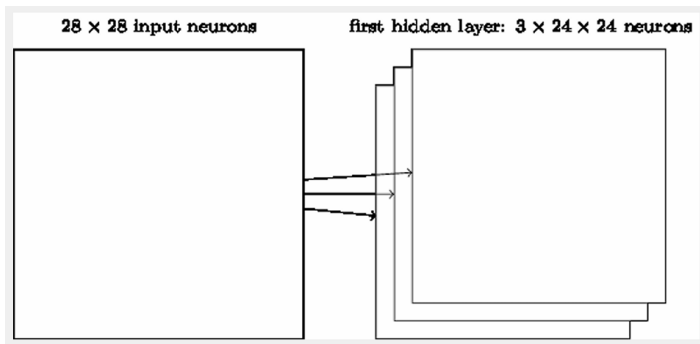
Shared Weights and Biases

- We sometimes call the map, from the input layer to the hidden layer, a **feature map**.
 - Suppose **the weights and bias** are such that the hidden neuron can **pick out a feature** (e.g., a vertical edge) in a particular local receptive field.
 - That ability is also likely to be useful at other places in the image.
 - And therefore it is useful to apply **the same feature detector** everywhere in the image.
- We call the weights and bias defining the feature map the **shared weights** and the **shared bias**, respectively.
- A set of the shared weights and bias defines clearly **a kernel or filter**.

To put it in slightly more abstract terms, CNNs are well adapted to the **translation invariance** of images.

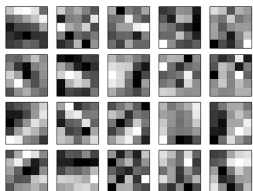
Multiple Feature Maps

- The network structure we have considered so far can detect just **a single localized feature**.
- To do **more effective image recognition**, we'll need more than one **feature map**.
- Thus, a complete convolutional layer consists of **several different feature maps**:



Multiple Feature Maps

Modern CNNs are often built with 10 to 50 feature maps, each associated to a $r \times r$ local receptive field: $r = 3 \sim 9$.



The 20 images corresponding to 20 different feature maps, which are actually learned when classifying the MNIST dataset ($r = 5$).

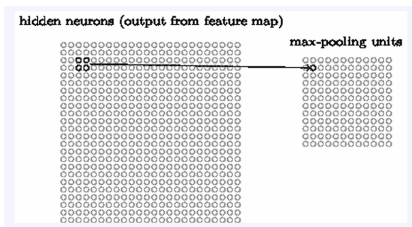
The number of parameters to learn

A big advantage of sharing weights and biases is that it greatly reduces the number of parameters involved in a convolutional network.

- Convolutional networks: $(5 \times 5 + 1) * 20 = 520$
- Fully-connected networks: $(28 \times 28 + 1) * 20 = 15,700$

Pooling

- CNNs also contain **pooling layers**, in addition to the convolutional layers just mentioned.
- Pooling layers are usually used **right after convolutional layers**.
- What they do is **to simplify the information** in the output from the convolutional layer.

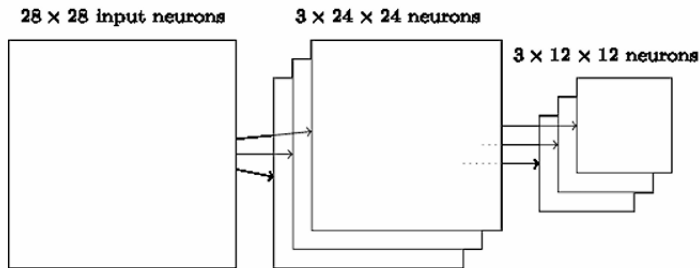


Pooling

Summarizing a region of 2×2 neurons in the convolutional layer.

Pooling

Since we have 24×24 neurons output from the convolutional layer, after pooling we will have 12×12 neurons for each feature map:



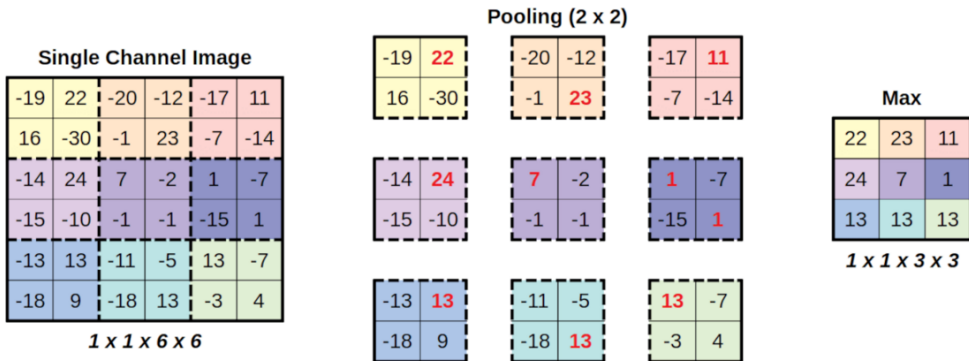
Types of Pooling

1. **max-pooling:** simply outputs the maximum activation in the 2×2 input neurons.
2. **L^2 -pooling:** outputs the L^2 -average of the 2×2 input neurons.

Pooling

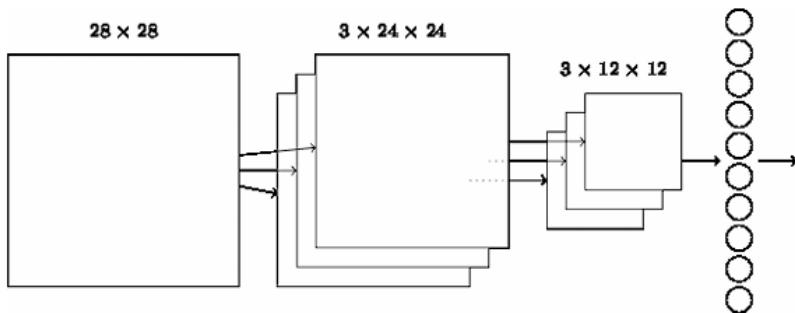
Types of Pooling

Worked example of max pooling, with filter size $f = 2$ and stride $s = 2$.



CNNs, in practice

A simple CNN of three feature maps, to classify MNIST digits.



CNNs, in practice

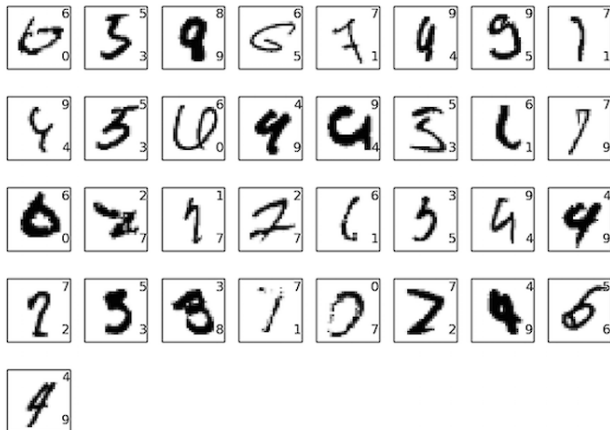
- To form a complete CNN by putting all these ideas, we need to **add some extra layers**, below the convolution-pooling layers.
 - Figure in Slide 17 shows a CNN that involves an extra layer of 10 output neurons, for the 10 possible values for MNIST digits.
- **The final layer of connections** is a fully-connected layer. e.g.,:
 - Let `filter_shape = (20, 1, 5, 5)`, `poolsize = (2, 2)`
(20 feature maps; $1 \times 5 \times$ kernel; 2×2 pooling)
 - Then, the number of parameters to teach:
 $(5^2 + 1) \cdot 20 + (20 \cdot 12^2 + 1) \cdot 10 = 29,330$
 - Classification accuracy for the MNIST dataset $\lesssim 99\%$

CNNs, in practice

- Add a **second convolution-pooling layer**:
 - Its input is the output of the first convolution-pooling layer.
 - Let `filter_shape = (40, 20, 5, 5)`, `poolsize = (2, 2)`
 - The output of the second convolution-pooling layer: 40×4^2
 - Then, the number of parameters to teach:
 $(5^2 + 1) \cdot 20 + (5^2 + 1) \cdot 40 + (40 \cdot 4^2 + 1) \cdot 10 = 7,970$
 - Classification accuracy for the MNIST dataset $\gtrsim 99\%$
- Add a **fully-connected layer** (up the output layer):
 - Let choose 40 neurons: $\implies 27,610$ parameters
 - Classification accuracy $\approx 99.5\%$
- Use an **ensemble of networks**
 - Using 5 CNNs, classification accuracy = 99.67% (**33 missed!**)

CNNs, in practice

The images missed by an ensemble of 5 CNNs. The label in the top right is the correct classification, while in the bottom right is the label classified output.



Intuitively Speaking

- **(Better representation)**. The use of **translation invariance** by the convolutional layer will reduce the number of parameters it needs to get the same performance as the fully-connected model.
- **(Convolution kernels)**. The filters try to detect **localized features**, producing **feature maps**.
- **(Efficiency)**. **Pooling** simplifies the information in the output from the convolutional layer.
 - That, in turn, will result in **faster training** for the convolutional model, and, ultimately, will help us **build deep networks** using convolutional layers
- **Fully-connected hidden layers** try to collect information for **more** widely formed features.

Deep Learning Packages

- Keras

- A **high-level** ANN *application programming interface* (**API**).
- It can run on top of TensorFlow, Theano, and Microsoft Cognitive Toolkit (CNTK).
- Keras focuses on being modular, **user-friendly**, and extensible.
- Written in **Python**.

- TensorFlow

- **Low and High API levels:** An end-to-end open-source deep learning framework developed by **Google** in 2015
- It is a symbolic math library used for neural networks $\implies$ **fast!**
- Tensorflow is **difficult** to use and debug.
- Written in **Python, C++, CUDA**.

- **Pytorch**

- **Low and High API levels:** It is a deep learning framework based on Torch, developed by **Facebook** in 2017, and taken over by the PyTorch Foundation (part of Linux Foundation) in late 2022.
- It has **outstanding community support and development**.
- Pytorch is **much easier** to use and debug than Tensorflow.
- Written in **Lua**.

You do not need to learn C++ or Lua

- Currently, **Keras** is most popular due to its simplicity and long history.
- **Pytorch** is most rapidly growing.
 - It can be viewed as a **trade-off** between Keras and Tensorflow.
 - It is particularly good for **natural language processing applications**.
 - **Mathematicians and experienced researchers** will find Pytorch better than Keras, for many applications.



Cluster Analysis

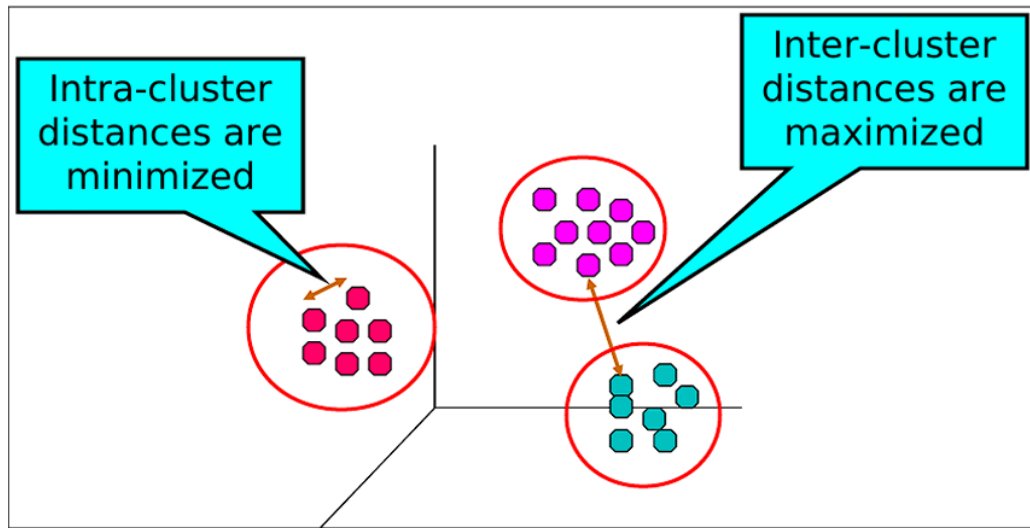
Cluster Analysis

Cluster analysis or **clustering** is **the task of finding groups of objects** such that the objects in a group will be similar (or related) to one another and different from (or unrelated to) the objects in other groups. It is a main task of **exploratory data mining**, and a common technique for **statistical data analysis**, used in many fields, including machine learning, pattern recognition, image analysis, information retrieval, bioinformatics, data compression, and computer graphics.

History

Cluster analysis was originated in **anthropology** by Driver and Kroeber in 1932, introduced to **psychology** by Zubin in 1938 and Robert Tryon in 1939, and famously used by Cattell beginning in 1943 for trait theory classification in **personality psychology**.

Basics for Cluster Analysis



- **Understanding**
 - group related documents or browsings
 - group genes/proteins that have similar functionality, or
 - group stocks with similar price fluctuations
- **Summarization**
 - reduce the size of large datasets

Not Cluster Analysis

- Supervised classification – Uses class label information
- Simple segmentation – Dividing students into different registration groups alphabetically, by last name
- Results of a query – Groupings are a result of an external specification

History

Clustering uses **only the data** (unsupervised learning): to discover **hidden structures** in data.

Quality of Clustering

- A **good clustering** method will produce high quality clusters with:
 - high intra-class similarity
 - low inter-class similarity
- The quality of a clustering result depends on both **the similarity measure** and **its implementation**
- The quality of a clustering method is also measured by its ability to discover some or all of the **hidden patterns**

Measuring the Quality of Clustering

- Dissimilarity/Similarity/Proximity metric: Similarity is expressed in terms of a distance function $d(i,j)$.
- The definitions of **distance functions** are usually very different for interval-scaled, boolean, categorical, ordinal ratio, and vector variables.
- There is a separate “quality” function that measures the “goodness” of a cluster.
- *Weighted measures*: Weights should be associated with different variables based on applications and data semantics.
- It is hard to define “similar enough” or “good enough”
 - the answer is typically highly subjective

Similarity and Dissimilarity Between Objects

- **Distances** are normally used to measure the similarity or dissimilarity between two data objects.
- Some popular ones include: **Minkowski distance**

$$d(i, j) = \|\mathbf{x}_i - \mathbf{x}_j\|_p = (|x_{i1} - x_{j1}|^p + |x_{i2} - x_{j2}|^p + \dots + |x_{id} - x_{jd}|^p)^{1/p},$$

where $\mathbf{x}_i, \mathbf{x}_j \in \mathbb{R}^d$, two d -dimensional data objects.

- When $p = 1$, it is **Manhattan distance**
- When $p = 2$, it is **Euclidean distance**

Similarity and Dissimilarity Between Objects

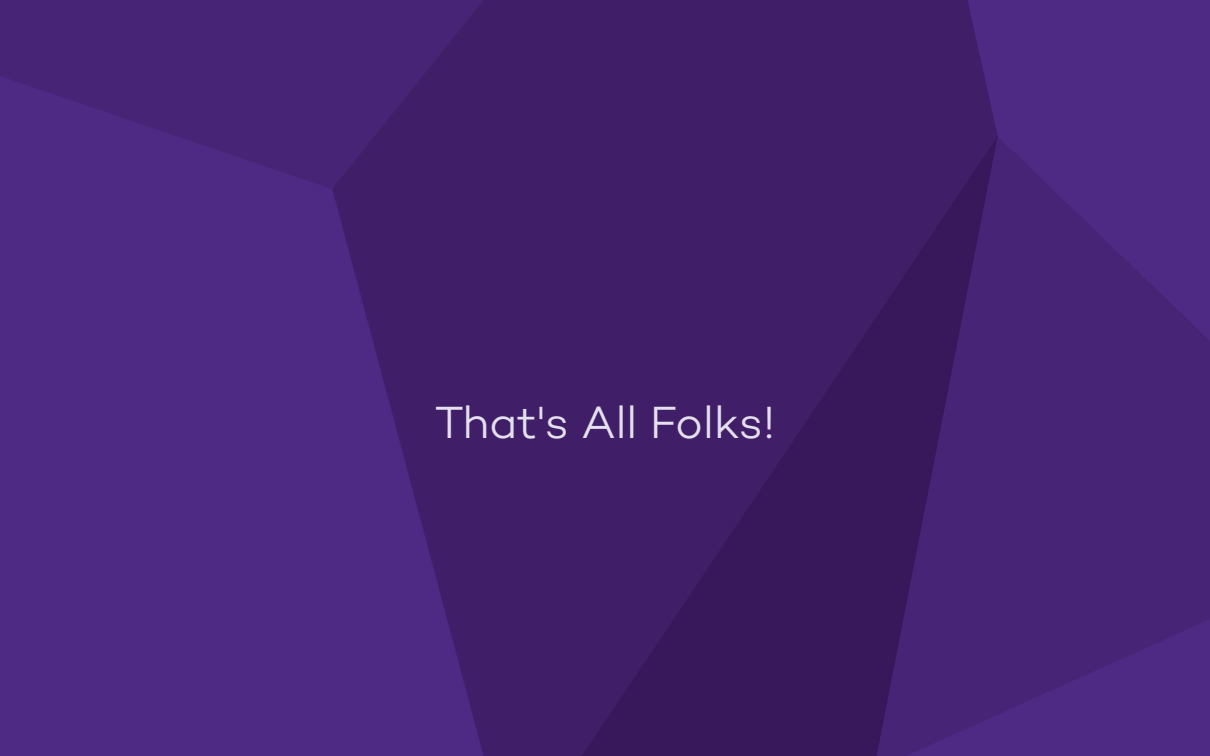
- *Other Distances*: Also, one can use weighted distance, parametric Pearson product moment correlation, or other dissimilarity measures.
- Various similarity measures have been studied for:
 - Binary variables
 - Nominal variables & ordinal variables
 - Ratio-scaled variables
 - Variables of mixed types
 - Vector objects

Types of Cluster

- Center-based clusters
- Contiguity/connectivity-based clusters
- Density-based clusters
- Conceptual clusters

Well-separated Clusters

A cluster is a set of objects such that an object in a cluster is closer (more similar) to **every/some of points** in the cluster, than any points not in the cluster.

The background is composed of several overlapping, semi-transparent purple polygons of various shapes and sizes, creating a complex, layered geometric pattern. The colors range from a deep, dark purple to a slightly lighter, more vibrant shade.

That's All Folks!